# Day 75 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 11, Day 6 reference

Watch: Why Scaling by the Square Root of Dimensions Matters in Attention (Learn With Jay) | Self-Attention Using Scaled Dot-Product Approach (Machine Learning Studio)

`day75_scaled_dot_product_attention.py`, segment by segment, in the order `main()` uses things. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

**MATH BOX LEGEND**

**Symbols:** d = length of query and key vectors · c = the factor in front of q · k (1, 1/√d or 1/d) · spread = standard deviation · entropy H = – Σ a log a

## Segment 0 — Setup

### Settings and the softmax pieces

**CODE**

```
N_KEYS = 32
N_PAIRS = 16
SEEDS = [0, 1, 2]
STEPS = 2000
CHECKS = (100, 250, 500, 1000, 2000)
DIMS = (16, 64, 128)
def softmax(z):
    e = np.exp(z - z.max(axis=-1, keepdims=True))
    return e / e.sum(axis=-1, keepdims=True)
def softmax_backward(dA, A):
    return A * (dA - (A * dA).sum(axis=-1, keepdims=True))
```

- `CHECKS = (100, 250, ...)` — a tuple of steps at which the test accuracy is recorded.
- `softmax_backward(dA, A)` — Day 70 exercise's formula as its own function, reused in Sections 3 and 4.

## Segment 1 — The argument, step by step

### The spread of q . k

**MATH**

$$Var(q \cdot k) = d \quad \Rightarrow \quad spread \sqrt{d}$$

**CODE**

```
for d in [4, 16, 64, 256, 1024]:
    q, k = rng.randn(10000, d), rng.randn(10000, d)
    s = (q * k).sum(axis=1)
    print(f"{d:>6}{s.std():>17.3f}{np.sqrt(d):>9.3f}{(s / np.sqrt(d)).std():>27.3f}")
```

- `(q * k).sum(axis=1)` — 10,000 dot products at once: multiply element by element, then sum each row.
- `s.std()` — the spread of the 10,000 values.

### Softmax and gradient at each size

MATH

$$dS_{raw} = c \cdot A \times (dA - \Sigma\, A\, dA), \quad A = softmax(c\, S_{raw})$$

CODE

```
dA = np.random.RandomState(3).randn(5000, 16)
r = []
for c in [1.0, 1 / np.sqrt(d)]:
    dS = softmax_backward(dA, softmax(c * S)) * c
    size = np.linalg.norm(dS, axis=1)
    r += [np.median(size), (size < 1e-6).mean()]
```

- `np.random.RandomState(3).randn(...)` — a fresh rng with the same seed for every size: the same upstream gradient each time.
- `softmax_backward(dA, softmax(c * S)) * c` — back through the softmax, then through the factor c, to the raw q . k.
- `np.linalg.norm(dS, axis=1)` — the size of each query's gradient vector.
- `(size < 1e-6).mean()` — a boolean array's mean is the share of True values.

## Segment 2 — The recall model

### make_batch: associative recall

CODE

```
def make_batch(rng, m):
    keys = np.array([rng.permutation(N_KEYS)[:N_PAIRS] for _ in range(m)])
    vals = rng.randint(0, N_KEYS, size=(m, N_PAIRS))
    pick = rng.randint(0, N_PAIRS, size=m)
    return keys, vals, keys[np.arange(m), pick], vals[np.arange(m), pick], pick
```

- `rng.permutation(N_KEYS)[:N_PAIRS]` — 16 different keys: the first 16 of a shuffled 0..31.
- `keys[np.arange(m), pick]` — fancy indexing: in example i take column pick[i].
- `five return values` — pairs' keys and values, the query key, the answer, and where it sits.

### Recall.__init__ and forward

**MATH**

$$a = softmax(\, c\,(x_q\, W_q) \cdot (x_j\, W_k)\,);\quad ctx = \Sigma\, a_j\, x_j\, W_v$$

**CODE**

```
class Recall:
    def __init__(self, d, scale="sqrt", seed=0, lr=0.1, clip=1.0, qk_init=1.0):
        rng = np.random.RandomState(seed)
        self.d, self.lr, self.clip = d, lr, clip
        self.c = {"none": 1.0, "sqrt": 1 / np.sqrt(d), "d": 1 / d}[scale]
        self.P = dict(Ek=rng.randn(N_KEYS, d),
                      Ev=rng.randn(N_KEYS, d),
                      Eq=rng.randn(d),
                      Wq=rng.randn(d, d) / np.sqrt(d) * qk_init,
                      Wk=rng.randn(d, d) / np.sqrt(d) * qk_init,
                      Wv=rng.randn(d, d) / np.sqrt(d),
                      Wo=rng.randn(d, N_KEYS) / np.sqrt(d), bo=np.zeros(N_KEYS))
    def forward(self, keys, vals, qkey):
        P = self.P
        X = P["Ek"][keys] + P["Ev"][vals]
        xq = P["Ek"][qkey] + P["Eq"]
        q, K, Vv = xq @ P["Wq"], X @ P["Wk"], X @ P["Wv"]
        S = self.c * np.einsum("md,mnd->mn", q, K)
        A = softmax(S)
        ctx = np.einsum("mn,mnd->md", A, Vv)
        p = softmax(ctx @ P["Wo"] + P["bo"])
        self.cache = dict(keys=keys, vals=vals, qkey=qkey, X=X, xq=xq, q=q, K=K, V=Vv, S=S, A=A,
            ctx=ctx, p=p)
        return p
```

- `{"none": 1.0, "sqrt": 1 / np.sqrt(d), "d": 1 / d}[scale]` — a dictionary used as a lookup table: the factor for each setting.
- `qk_init=1.0` — multiplies the initial Wq and Wk; Section 6 uses 64^(-1/4).
- `P["Ek"][keys] + P["Ev"][vals]` — each pair as one vector: the key's embedding plus the value's.
- `np.einsum("md,mnd->mn", q, K)` — one query against 16 keys, per example.

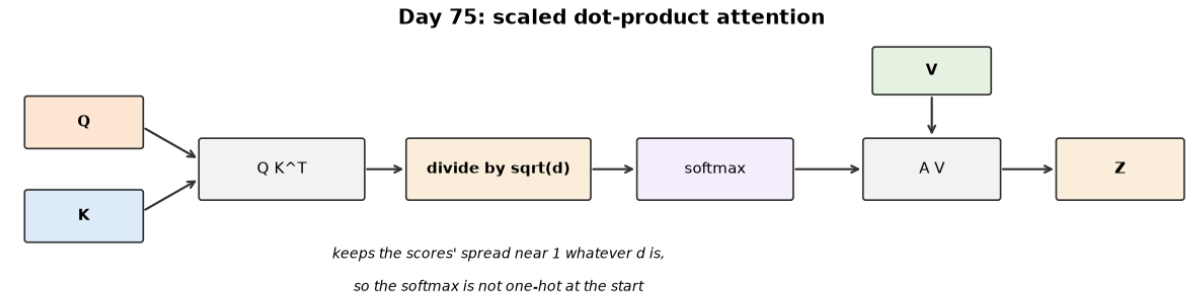
## loss_and_grads

**CODE**

```
def loss_and_grads(self, keys, vals, qkey, y):
    P = self.P
    p = self.forward(keys, vals, qkey)
    C = self.cache
    m = len(y)
    loss = -np.log(p[np.arange(m), y] + 1e-12).mean()
    dl = p.copy(); dl[np.arange(m), y] -= 1; dl /= m
    g = dict(Wo=C["ctx"].T @ dl, bo=dl.sum(axis=0))
    dctx = dl @ P["Wo"].T
    dA = np.einsum("md,mnd->mn", dctx, C["V"])
    dV = C["A"][:, :, None] * dctx[:, None, :]
    dS = softmax_backward(dA, C["A"]) * self.c
    dq = np.einsum("mn,mnd->md", dS, C["K"])
    dK = dS[:, :, None] * C["q"][:, None, :]
    flat = lambda T: T.reshape(-1, T.shape[-1])
    g["Wq"] = C["xq"].T @ dq
    g["Wk"] = flat(C["X"]).T @ flat(dK)
    g["Wv"] = flat(C["X"]).T @ flat(dV)
    dX = dK @ P["Wk"].T + dV @ P["Wv"].T
    dxq = dq @ P["Wq"].T
    g["Ek"], g["Ev"] = np.zeros_like(P["Ek"]), np.zeros_like(P["Ev"])
    np.add.at(g["Ek"], C["keys"], dX); np.add.at(g["Ev"], C["vals"], dX)
    np.add.at(g["Ek"], C["qkey"], dxq); g["Eq"] = dxq.sum(axis=0)
    return loss, g
```

- `softmax_backward(dA, C["A"]) * self.c` — the only place the scaling appears in the backward pass.
- `np.add.at(g["Ek"], C["keys"], dX)` — the same key embedding used by many pairs collects all their gradients.
- `np.add.at(g["Ek"], C["qkey"], dxq)` — the query's key embedding is the SAME table: it collects from the query too.

**SYSTEM DIAGRAM**




*Scaled dot-product attention as a pipeline.*

# Segment 3 — Experiments

## train and the caches

CODE

```
def train(d, scale, seed, qk_init=1.0, steps=STEPS):
    net = Recall(d, scale, seed=seed, qk_init=qk_init)
    rng = np.random.RandomState(seed + 100)
    curve = []
    start = time.perf_counter()
    for step in range(1, steps + 1):
        keys, vals, qkey, y, _ = make_batch(rng, 64)
        net.train_step(keys, vals, qkey, y)
        if step in CHECKS:
            curve.append(accuracy(net))
    return net, np.array(curve), time.perf_counter() - start
RUNS = {}
def train_cached(*args, **kwargs):
    key = (args, tuple(sorted(kwargs.items())))
    if key not in RUNS:
        RUNS[key] = train(*args, **kwargs)
    return RUNS[key]
```

- `if step in CHECKS:` — membership test in a tuple.
- `time.perf_counter() - start` — seconds for the whole run, kept but not reported (Day 69).
- `RUNS[key]` — Sections 4, 5 and 6 share runs: each configuration trains once.

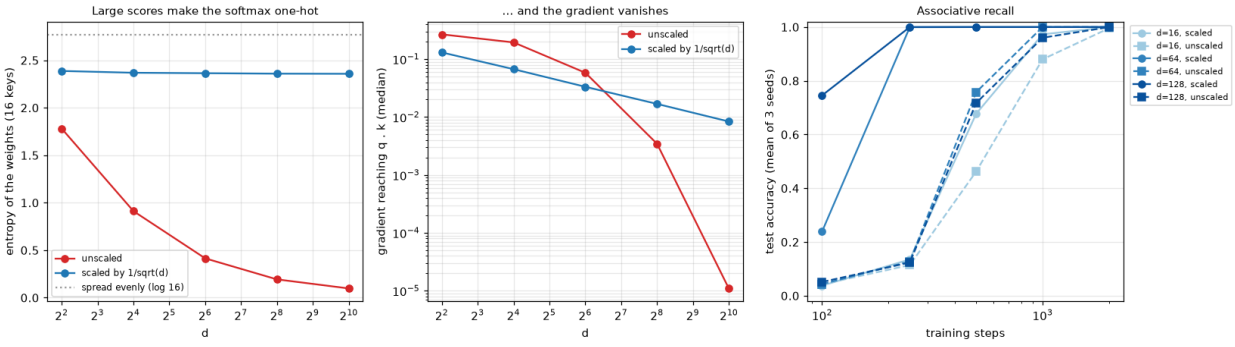
## Scaling or a smaller start

CODE

```
shrink = 64 ** -0.25
a, b = Recall(64, "sqrt", seed=0), Recall(64, "none", seed=0, qk_init=shrink)
pa, pb = a.forward(*TEST[:3]), b.forward(*TEST[:3])
```

- `64 ** -0.25` — a power with a negative fraction: 1 / 64^(1/4) = 1/2.83.
- `*TEST[:3]` — unpack the first three items of the test tuple as three arguments.
- `np.abs(a.cache['A'] - b.cache['A']).max()` — the two models' starting weights compared entry by entry.

REAL GRAPH




*Real graph from the script: softmax entropy, gradient size, and recall training.*

# Quick Syntax Glossary — new or reinforced constructs

- `{"a": 1, "b": 2}[key]` — a dictionary as a lookup table.
- `x ** -0.25` — a negative fractional power.
- `f(*t)` — unpack a tuple into positional arguments.

- `np.linalg.norm(A, axis=1)` — the length of every row.
- `(cond).mean()` — the share of entries where a condition holds.
- `rng.permutation(n)[:k]` — k different integers from 0..n-1.
- `ax.set_xscale("log", base=2)` — a log axis with powers of 2.

*End of Day 75 syntax reference. For the experiments and what they showed, see the Day 75 complete guide.*